CODEBASE REVIEW / EXECUTIVE EDITION

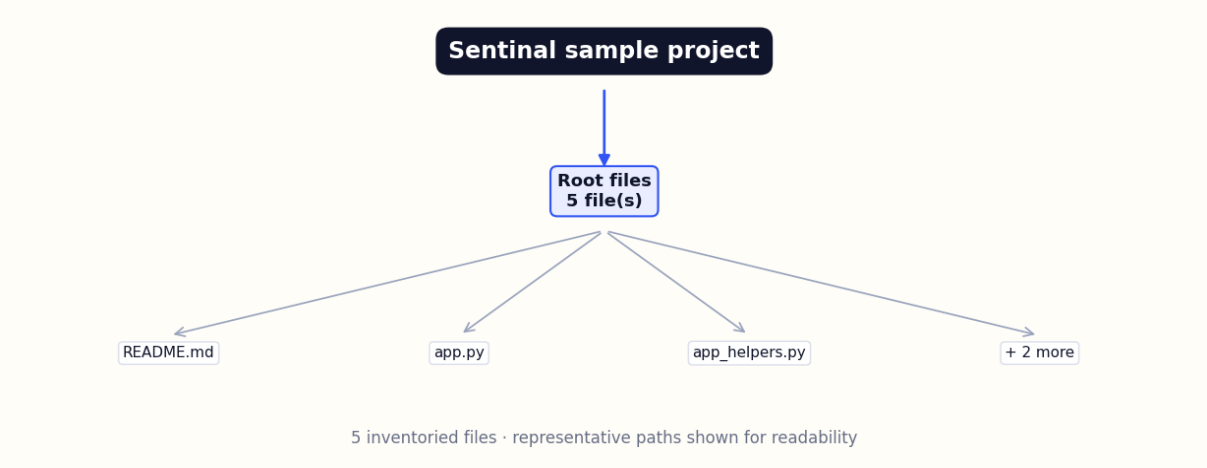
Know what to fix next.

A compact, evidence-led snapshot of structure, risk signals, and practical next steps.

PROJECT	Sentinal sample project	SCAN TIME (UTC)	2026-10-18 09:01:38 UTC
SOURCE	Sample / uploaded source	BRANCH	demo
REVISION	local-demo-fixture	REPORT VERSION	2.0

01 / REPOSITORY SHAPE

The hierarchy below groups the inventoried paths by their top-level folder. Representative files are shown so the structure stays readable at presentation size.



FILES SCANNED	SOURCE LINES	FINDINGS	HEALTH TRIAGE
5	25	8	70/100

Reading this report

Sentinal reports deterministic pattern matches and static relationships. A reported signal needs contextual review; files without a reported signal are not certified safe. Submitted application code is not executed by default.

INVENTORY / COVERAGE

What was actually scanned

FILES SCANNED	SOURCE LINES	FINDINGS	HEALTH TRIAGE
5	25	8	70/100

File-type inventory

1 dependency manifest file(s) and 0 likely test file(s) were detected by available heuristics.

FILE TYPE	FILES	EXAMPLE
.py	2	app.py
.js	1	service.js
.txt	1	requirements.txt
.md	1	README.md

ADDITIONAL MEASURES	VALUE
Python parse errors	0
Functions detected	4
Classes detected	0
Physical lines	43

Files to review first

FILE / LOCATION	SIGNALS	HIGHEST SEVERITY
app.py	3	HIGH
service.js	2	HIGH

No current file-scoped finding

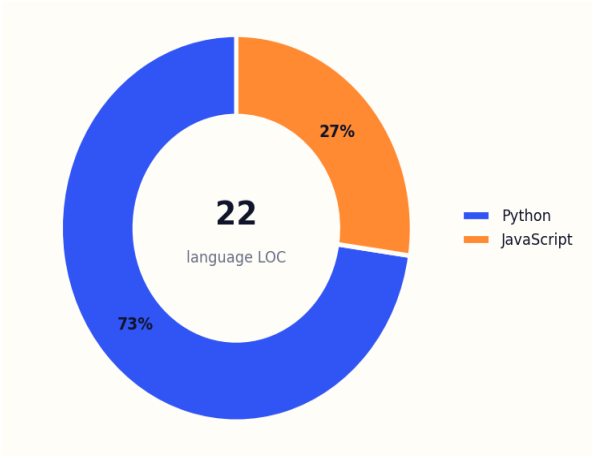
app_helpers.py, README.md

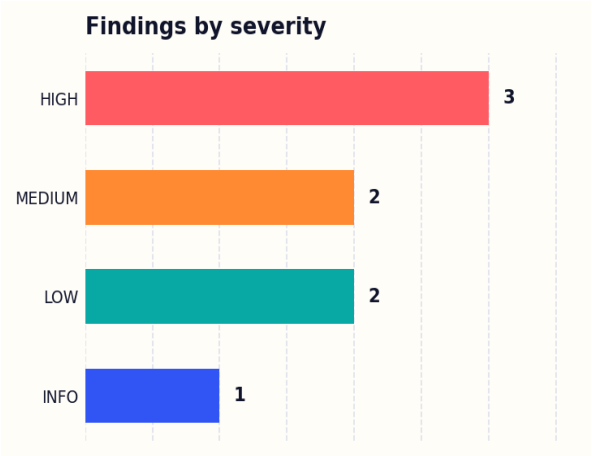
This list means only that the enabled checks did not attach a finding to those paths. It does not establish that they are safe or defect-free.

SIGNALS / VISUAL SUMMARY

The shape of the codebase

Language share is based on the scanner's non-comment, non-blank line estimate. Severity counts are rule-based findings; they are not vulnerability confirmations.





What stands out

- 2 language group(s) identified; the leading group by measured lines is Python
- 3 high/critical signal(s) and 2 medium signal(s) deserve early human review.
- Health triage is 70/100; it is a confidence-weighted heuristic, not a security certification.

Language breakdown

LANGUAGE	FILES	LINES
Python	2	16
JavaScript	1	6

DECISION SUPPORT / REMEDIATION

A practical review queue

Sentinal sample project contains 5 analyzed text files and 25 non-comment/non-blank source lines. The deterministic checks reported 8 finding(s), including 3 high/critical item(s). Findings are rule-based signals that should be confirmed in context; this scan did not execute project code or query a live advisory database.

LEVEL	FINDING / LOCATION	WHY IT MATTERS	NEXT ACTION
HIGH	**32BF782D4BA7 · Subprocess uses shell=True** app.py:18	Shell interpolation can turn untrusted input into operating-system commands.	Pass an argument list with shell=False and validate each argument.
HIGH	**BCF78007EBAB · Dynamic code execution via eval/exec** app.py:13	User-controlled text passed to eval/exec can execute arbitrary Python code.	Replace dynamic execution with explicit parsing or a constrained mapping of allowed operations.
HIGH	**68D27E2455D9 · Dynamic JavaScript evaluation** service.js:8	eval or Function constructors can execute attacker-controlled code.	Replace dynamic evaluation with structured data handling and explicit dispatch.
MEDIUM	**98F03FE55E5E · Potential unsafe YAML load** app.py:23	yaml.load without a safe loader may construct arbitrary Python objects.	Use yaml.safe_load or explicitly configure SafeLoader for untrusted YAML.
MEDIUM	**CF5BB1314E44 · Unsafe HTML injection sink** service.js:4	Assigning untrusted strings to innerHTML can enable cross-site scripting.	Prefer textContent or sanitize HTML with a well-maintained allow-list sanitizer.

Recommended next moves

- **Subprocess uses shell=True** — Shell interpolation can turn untrusted input into operating-system commands.
- **Dynamic code execution via eval/exec** — User-controlled text passed to eval/exec can execute arbitrary Python code.
- **Dynamic JavaScript evaluation** — eval or Function constructors can execute attacker-controlled code.

LIMITATIONS

Sample report for presentation preview.

No live dependency advisory database was queried. Dependency versions are reported only where detected in manifests; no advisory status is implied. Files with no attached finding have not been certified safe.